

Track C config (`configs/trackC.yaml`) — what each key changes (pruning eval)

Track C is **evaluation-only**: it takes a trained Track B checkpoint and applies **training-free RGTP-style token pruning** at inference time to trade accuracy for speed.

This note maps every key in `local_extraction/configs/trackC.yaml` to:

- **Where it's used** (`trackC_pruning.py` , dataset, TrackB modules)
- **What it changes** (metrics, pruning behavior, runtime)
- **Why it changes it** (mechanism in code)
- **How changing it changes results** (expected direction)

Written against:

- Main runner: `local_extraction/trackC/trackC_pruning.py`
- Underlying model parts (loaded from Track B checkpoint): `local_extraction/trackB/*`

How config is loaded

- Track C runner supports: `python -m trackC.trackC_pruning --config trackC`
- `trackC.yaml` inherits from `base.yaml` .
- Many model-related keys (`model.*`) are **not defined in** `trackC.yaml` , but the script uses safe defaults when they're missing.

Important: Track C does **not train** anything; it loads weights from a Track B checkpoint and optionally prunes candidates before running the head.

RGTP keys (`rgtp.*`) — pruning behavior (core Track C)

`rgtp.enabled`

- **Used by:** `trackC_pruning.py`
- **Effect:** enables pruning mask computation.
- **Why:** if disabled, all candidates are kept.
- **Changing it changes results by:**
 - `false` : baseline Track B inference (no pruning) → highest accuracy, slowest
 - `true` : pruning on → faster but may reduce accuracy

Note: CLI `--no_pruning` overrides this.

`rgtp.rate`

- **Used by:** `trackC_pruning.py`
- **Effect:** fraction of candidates pruned, capped to `[0.0, 0.95]` .
- **Why:** keep-count is computed from `num_candidates - round(num_candidates * rate)` .

- **Changing it changes results by:** higher rate → fewer candidates evaluated → faster, but higher chance you prune the true positive.

Note: CLI `--rgtp_rate` overrides this.

rgtp.min_keep

- **Used by:** `trackC_pruning.py`
- **Effect:** safety floor on how many candidates must remain.
- **Why:** prevents degenerate behavior when few candidates exist.
- **Changing it changes results by:** higher `min_keep` protects accuracy but reduces speed gains.

rgtp.temporal_decay

- **Used by:** `trackC_pruning.py`
- **Effect:** blends rollout saliency vs motion energy:

$$\text{grid_score} = \alpha \cdot \text{rollout} + (1 - \alpha) \cdot \text{motion}$$

where $\alpha = \text{temporal_decay}$.

- **Why:** rollout favors “what was attended to recently”; motion favors “what changed recently”.
- **Changing it changes results by:**
 - closer to `1.0` : prune mostly based on attention rollout (more stable, may miss fast-moving objects)
 - closer to `0.0` : prune mostly based on motion energy (can help for sudden interactions, can be noisy)

rgtp.logit_fill

- **Used by:** `trackC_pruning.py`
- **Effect:** sets logits for pruned candidates to a large negative number.
- **Why:** pruned candidates still exist in tensors; filling with e.g. `-12` makes their softmax probability near-zero.
- **Changing it changes results by:**
 - more negative: safer (pruned candidates almost never win)
 - less negative: pruned candidates may sometimes win due to numerical effects (usually not desired)

Evaluation keys (`evaluation.*`) — data + which checkpoint is evaluated

evaluation.checkpoint

- **Used by:** `trackC_pruning.py`
- **Effect:** which Track B checkpoint to load.
- **Why:** Track C uses Track B weights (projector, fusion, head).
- **Changing it changes results by:** different trained model → different baseline accuracy and pruning robustness.

CLI `--checkpoint` overrides this.

`evaluation.stageB_run`

- **Used by:** `trackC_pruning.py`
- **Effect:** chooses which Track A Stage B run provides `head_val.*`.
- **Why:** if null, it auto-detects latest Stage B run under `local_extraction/runs/Track_A`.
- **Changing it changes results by:** changing the validation dataset (different candidate proposals/crops/labels upstream).

`evaluation.val_manifest`

- **Used by:** `trackC_pruning.py`
- **Effect:** forces a specific validation manifest.
- **Why:** otherwise it resolves from StageB run (`head_val.*`).
- **Changing it changes results by:** changing samples/labels/candidate distribution.

`evaluation.batch_size`

- **Used by:** `trackC_pruning.py`
- **Effect:** evaluation throughput; typically `1` for “streaming-like” pruning.

`evaluation.candidate_limit`, `evaluation.normalize_ttc`

- **In YAML:** exists.
- **In code:** currently NOT read from `evaluation.*`.

Important nuance (current implementation):

- `EvalConfig.candidate_limit` reads `training.candidate_limit`.
- `EvalConfig.normalize_ttc` reads `training.normalize_ttc`.

So if you edit only `evaluation.candidate_limit` in `trackC.yaml`, nothing changes.

Workaround (no code changes): add a `training:` section to `trackC.yaml`:

```
training:
  candidate_limit: 16
  normalize_ttc: true
```

Optional re-ranking keys (not present in `trackC.yaml`, but supported)

Track C can optionally apply the same evaluation-time re-ranking as Track B eval. These keys are read from `evaluation.hotspot_priors.*` and `evaluation.clip_rerank.*`.

`evaluation.hotspot_priors.enabled` / `path` / `alpha`

- **Used by:** `trackC_pruning.py`
- **Effect:** blends candidate score with a (verb,noun) prior.
- **Default:** disabled.

CLI override:

- `--hotspot on|off`

`evaluation.clip_rerank.enabled / model / weight`

- **Used by:** `trackC_pruning.py`
- **Effect:** blends score with CLIP similarity of the crop to the predicted noun prompt.
- **Default:** disabled.

CLI override:

- `--clip on|off`

Note: CLIP requires installing the CLIP package; if it's not available, the script disables it automatically.

Instrumentation keys (`instrumentation.*`) — latency/VRAM/FLOPs

Track C collects latency (per sample), optional micro-benchmark timings, optional FLOPs, and CUDA peak memory.

Key mismatch warning

- `trackC.yaml` defines: `instrumentation.record_latency` , `record_vram` , `record_flops` , `warmup_iters` , `bench_iters` , `bench_samples` .
- `trackC_pruning.py` currently reads different keys via `RuntimeConfig` :
 - `instrumentation.measure_latency` , `measure_vram` , `measure_flops`
 - `instrumentation.bench_warmup` , `bench_iters`

So, changing `instrumentation.record_latency` in YAML does **not** change behavior unless you also add the `measure_*` keys.

Practical workaround (no code changes): add these keys to `trackC.yaml` if you want control:

```
instrumentation:
  measure_latency: true
  measure_vram: true
  measure_flops: true
  bench_warmup: 1
  bench_iters: 5
```

Rate sweep (`rate_sweep.*`)

- In **YAML**: `rate_sweep.enabled` , `rate_sweep.rates` .
- In **`trackC_pruning.py`** : not used.

So enabling `rate_sweep` in YAML does nothing for the main runner; it's intended for separate sweep tooling (or future integration).

Output keys (`output.*`)

- In **YAML**: `output.runs_dir` , `metrics_subdir` , `plots_subdir` .
- In **code**: the runner currently writes metrics to:
 - `local_extraction/runs/Track_C/metrics/trackC_val_rateXX_*.json`

So changing `output.*` in YAML does not affect where metrics are written.

CLI overrides (fast experiments)

Supported in `trackC_pruning.py` :

- `--video_backbone resnet18|videomae_ego` (forces tokenizer backbone during eval)
 - `--rgtp_rate <float>` (overrides pruning rate)
 - `--no_pruning` (baseline mode)
 - `--checkpoint <path>` (evaluate specific checkpoint)
 - `--hotspot on|off` , `--clip on|off`
-

Summary (what to change for different outcomes)

High-impact accuracy vs speed knobs:

- `rgtp.rate` : main Pareto knob (more pruning → faster but less accurate)
- `rgtp.min_keep` : safety knob (protect accuracy)
- `rgtp.temporal_decay` : what the pruning "trusts" (rollout vs motion)

High-impact "what you are evaluating" knobs:

- `evaluation.checkpoint` : which Track B model you're pruning
- `evaluation.stageB_run` / `evaluation.val_manifest` : which validation set (from Track A Stage B) you're using

Profiling knobs:

- instrumentation toggles (but note the current YAML/key mismatch)
-

Numerical examples (illustrative comparisons)

These are directional examples showing how settings typically behave.

Example 1 — Pruning rate vs kept candidates

If a frame has 20 candidates:

<code>rgtp.rate</code>	Expected keep count (approx)	Expected effect
0.0	20	baseline accuracy, slowest
0.1	18	small speedup, tiny accuracy drop
0.3	14	medium speedup, moderate accuracy drop
0.5	10	large speedup, higher risk pruning GT
0.7	6	very fast, often unstable unless GT is consistently salient

Example 2 — `min_keep` safety floor

Assume `rgtp.rate=0.7` and 10 candidates exist.

<code>min_keep</code>	Keep count (min)	Expected effect
1	3	fastest but risky
2	3	default-like safety
5	5	better accuracy, less speedup

Example 3 — `temporal_decay` blending

Assume rollout score is strong for stationary objects and motion score is strong for moving objects.

<code>temporal_decay</code>	Pruning preference	Typical failure mode
0.9	rollout-attended tokens	misses sudden/fast interactions
0.6	balanced (default)	generally stable
0.2	motion-driven tokens	noisy on camera motion / background movement

Example 4 — `logit_fill` effect

After pruning, dropped candidates get filled logits.

<code>logit_fill</code>	Softmax impact on dropped candidates	Practical note
-6	may still get non-trivial prob	not recommended
-12	near-zero prob	good default
-20	essentially impossible	safest (no pruned candidate wins)